Project 1: Fine-Tuning LLaMA Model for Address Extraction

Objective

The primary goal of this project was to develop a fine-tuned LLM capable of accurately extracting address information from unstructured and semi-structured legal documents. The model is intended to improve extraction accuracy, reduce dependence on manual processing, and highlight the effectiveness of large language models in automating address recognition across real-world legal texts.

Data Source

The dataset was derived from a variety of legal documents, including court proceedings, summons notices, and related files—typically in PDF format. These documents often lacked consistent formatting, making traditional machine learning approaches ineffective. To overcome this, we opted for a large language model (LLM) fine-tuning approach to understand and infer logical patterns in U.S. address structures, even in noisy or poorly organized data.

Data Collection and Preparation

1. **Text Extraction**
   AWS Textract was used to extract raw textual data from the PDF files. Due to the unstructured nature of the source documents, the extracted text lacked predictable patterns, which posed a challenge for conventional extraction methods.
2. **Entity and Context Extraction**
   Custom scripts were developed to identify defendant names using a combination of:
   - Exact string matches
   - Fuzzy matching
   - Similarity-based heuristics

   Once a name was identified, we extracted a 300–500 word window following the name, which likely included the corresponding address.

3. **Labeling**
   Human annotators labeled the correct address from the extracted passages. This resulted in a dataset containing:
   - **Input:** Raw extracted context (including defendant names and nearby text)
   - **Output:** Cleaned and validated address information

4. **Formatting for Instruction Tuning**
   We fine-tuned the `meta-llama/Llama-2-13b-chat-hf` model using instruction tuning techniques. The training data was structured using Meta's recommended **Jinja-based chat template** to simulate conversational input-output pairs.

Jinja Prompt Formatting Template

We followed the official `chat_template` for LLaMA-2 instruction tuning, which uses role-based structuring (`user`, `assistant`, `system`). The format ensures that the input prompt and expected response are encoded in a predictable, instruction-tuned format.

Here is the template we used:

```
{%- if messages[0]['role'] == 'system' -%}
  {%- set loop_messages = messages[1:] -%}
  {%- set system_message = messages[0]['content'] -%}
{%- else -%}
  {%- set loop_messages = messages -%}
  {%- set system_message = false -%}
{%- endif -%}

{%- for message in loop_messages -%}
  {%- if (message['role'] == 'user') != (loop.index0 % 2 == 0) -%}
    {{ raise_exception('Conversation roles must alternate
user/assistant/user/assistant/...') }}
  {%- endif -%}

  {%- if loop.index0 == 0 and system_message != false -%}
    {%- set content = '<<SYS>>\n' + system_message + '\n<</SYS>>\n\n' +
message['content'] -%}
  {%- else -%}
    {%- set content = message['content'] -%}
  {%- endif -%}

  {%- if message['role'] == 'user' -%}
    {{ bos_token + '[INST] ' + content.strip() + ' [/INST]' }}
  {%- elif message['role'] == 'assistant' -%}
    {{ ' ' + content.strip() + ' ' + eos_token }}
  {%- endif -%}
{%- endfor -%}
```

Instruction-Tuned Prompt Examples

To guide the model in generating accurate and professional address extractions, we incorporated clear instruction prompts. Below are examples of system and user prompts used in training:

System Prompt (for tone and behavior alignment):

```
<<SYS>>
You are a highly capable legal assistant. Your task is to extract and
return only the defendant's full address from messy legal text. Respond
clearly and concisely.
<</SYS>>
```

User Prompt Example:

```
[INST] Extract the defendant’s full address from the following text:

"Mr. John D. Miller is hereby summoned to appear before the County
Court. The case number 23-CV-1182 will be held at the Riverside Court,
CA. The legal notice has been sent to 578 Greenhill Road, Apt 4B,
Westfield, IL 62704. Please note that..."

 [/INST]
```

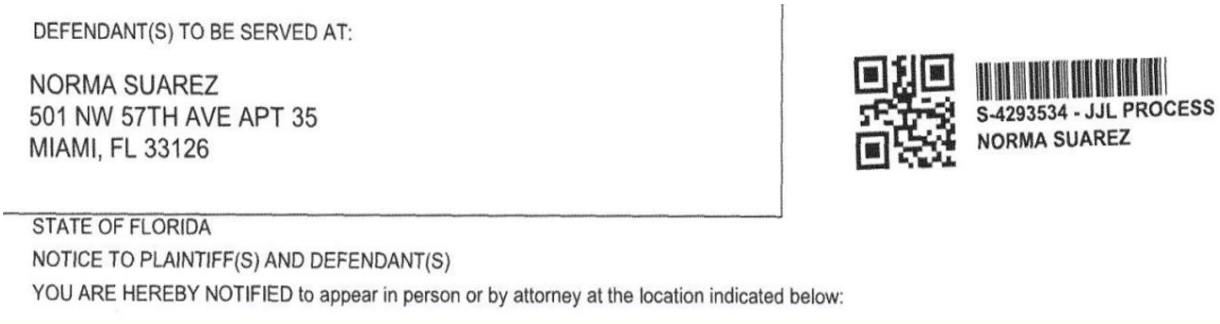
Assistant Output (Completion):

```
578 Greenhill Road, Apt 4B, Westfield, IL 62704 </s>
```

Training Summary

- **Model Used:** `meta-llama/Llama-2-13b-chat-hf` (instruction-tuned variant)
- **Training Format:** Prompt-completion pairs using the Jinja `chat_template`
- **Token Insertion:** Special tokens `<s>` (start) and `</s>` (end) were used to mark the boundaries of assistant completions.
- **Training Method:** Supervised fine-tuning via Hugging Face Trainer with a conversational dataset in the form of the prompt completion.

Problem it Solved:


DEFENDANT(S) TO BE SERVED AT:

NORMA SUAREZ
501 NW 57TH AVE APT 35
MIAMI, FL 33126

S-4293534 - JJL PROCESS
NORMA SUAREZ

STATE OF FLORIDA
NOTICE TO PLAINTIFF(S) AND DEFENDANT(S)
YOU ARE HEREBY NOTIFIED to appear in person or by attorney at the location indicated below:

AWS Textract Extracted Text - NORMA SUAREZ DEFENDANT(S) TO BE SERVED AT: NORMA SUAREZ 501 NW 57TH AVE APT 35 S- 4293534 JJL

PROCESS MIAMI, FL 33126 NORMA SUAREZ STATE OF FLORIDA NOTICE TO PLAINTIFF(S) AND DEFENDANT(S) YOU

Correct Address - 501 NW 57TH AVE APT 35 MIAMI, FL 33126

Address fetch by Python Library - 501 NW 57TH AVE APT 35 S-4293534 JJL PROCESS MIAMI, FL 33126

Address fetch by ML model - 501 NW 57TH AVE APT 35 S-4293534 JJL PROCESS MIAMI, FL 33126

Address fetch by LLM AI Model - 501 NW 57TH AVE APT 35 MIAMI, FL 33126

Project 2: Fine-Tuning OpenAI Model on Custom Cleaning dataset.

Project: - Our objective is to enhance OpenAI's GPT-3 model by fine-tuning it using a healthcare dataset, specifically a dataset from US clinics that includes queries, symptoms, and treatment plans. Subsequently, we intend to integrate the fine-tuned model with the Rasa Framework and deploy the system on WhatsApp. The project aims to develop a chatbot capable of generating treatment plans based on user-provided symptoms.

Data Source :- Utilize a medical dataset from a US clinic, incorporating ,queries, symptoms, and treatment plans, to enhance GPT-3's understanding of healthcare contexts.

Dataset Preparation and Training Process Overview

Dataset Structure:
The dataset was structured in a question-answer format and included the following fields:

- **Query:** Describing the patient's concern or medical question.
- **Symptoms:** A list of symptoms reported by the patient.
- **Treatment Suggestions:** Clinical guidance generated by the Clinical experts, outlining treatment plans, recommended steps, and initial procedures as suggested by certified medical professionals.

The data was originally curated and validated by clinical experts, ensuring high reliability and relevance. As a result, minimal preprocessing was required on our part. We focused primarily on removing duplicate entries and performing light data cleaning. This included standardizing training examples by stripping unnecessary whitespace—particularly important given the voice-to-text data collection method, which had introduced formatting inconsistencies.

Instruction Injection:
To align the model's output with the expected tone and professionalism of clinical communication, we injected standardized instruction prompts. These prompts guided the model to respond politely and maintain a formal, professional tone consistent with that used by healthcare practitioners. The instructional text was included as part of the system prompt, alongside the user query and symptoms, to ensure consistent behavior during generation.

- *"You are a professional clinical assistant. Respond politely and provide clear, concise treatment suggestions in line with standard clinical practices."*

- *"Act as a certified medical advisor. Ensure the response is respectful, structured, and aligned with professional healthcare communication standards."*

- *"Please provide a treatment plan with an empathetic and professional tone suitable for patient communication."*

Training Process:
We utilized OpenAI's Fine-tuning API, adopting the standard prompt-completion training format. The dataset was converted into `jsonl` files, each containing structured prompt-completion pairs. Once the data was validated and formatted, a fine-tuning job was submitted using the API to adapt the base model to our clinical response task.